



date: August 23, 2002

to: CRMPC member companies

from: T.A.Baer, MS 0834, Org. 9114

subject: Solution procedure for three dimensional free surface flow and 3D remeshing (GT-024.0)

keywords: 3D, free surface flows, ROT conditions, iterative solver, convergence rate, mesh, solution procedure, scaling, pressure stabilization, remesh

input records: ROT, VELO_NORMAL, KINEMATIC

Introduction

Solution of fully three dimensional free surface flows is a unique *Goma* capability. However, these flow problems tend to be complex and accordingly the solution procedure is also a more complicated process. This memo will describe application of *Goma* to a three dimensional flow over a weir past a detachment point and the subsequent freely falling sheet. We will describe a reasonable meshing decomposition, the meshing itself, use of *Goma* to create important geometric features, finding the fixed mesh solution, and finally computing the moving mesh solution. We will also discuss strategies for improving the convergence rate of the iterative solvers, and the set of ROT conditions used as these are always the hardest part of any three dimensional free surface problem. We will conclude by described a procedure for remeshing a three-dimensional geometry with newly installed *Cubit* features.

Problem Description

The problem we set out to solve is depicted in Figure 1. Essentially it consists of a thick-walled trough with fluid welling up from the interior. The fluid source is not shown in the figure. The fluid spills over the top of the trough, flows down the sides and eventually detaches from the bottom and falls freely after that. Solid vertical boundaries are present at both edges of the trough to serve as edge guides to the fluid. At the exit, the fluid sheet is assumed to be drawn down at a specified rate.

The *Cubit* input journal file, the *Goma* input and material decks, and the *brk* input file constructed to solver this problem are available as digital files along with the mesh starting geometry and a set of final solution files. These files have been included on the *Goma* 4.0 distribution CD and are also available from the author. A list of the files in this package and a brief description is included with this document in Appendix A.

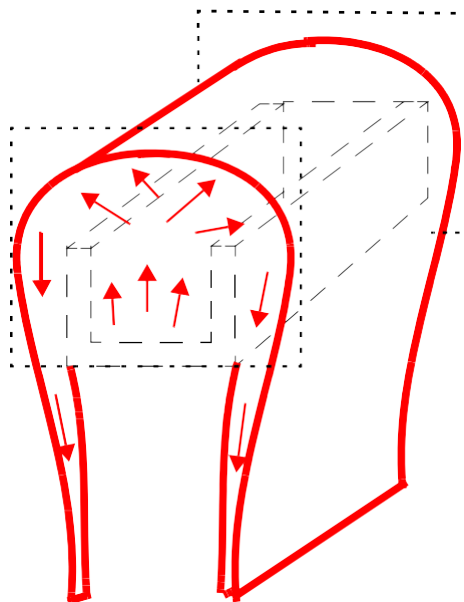


Figure 1. Sketch of the two-side flow over a weir geometry

Step 1 : Creating the mesh

The first step in any problem is creation of a meshed geometry. In three dimensions this step is always more involved. Indeed, for some problems it is the most difficult step. This is not the case here, but meshing this geometry does require some foresight and planning.

To begin with, the problem is symmetrical about a vertical plane through the centerline of the trough so only one-half the problem needs to be meshed. A second point is that it is always best to start with a smooth starting representation of the eventual free surface. That is, it is best that the starting set of surfaces that eventually will compose the free surface should not meet at angles less than 135° . This is a constraint imposed by the rotation algorithm employed by *Goma*, but it is something that must be kept in mind from the beginning.

The region where this constraint comes into play is at the edge of the freely falling film sheet where the free surface normal vector goes through a complete 180° transition from the front side of the free surface to the back side. While it is certainly an easier meshing problem if this transition region is a flat face (as suggested by the sketch) in which there is an abrupt 90° transitions from the front and reverse free surfaces, in practice this choice does not result in a very well-conditioned problem when solving with *Goma*. Instead, a better choice is a half-cylindrical surface connecting the front and reverse free surfaces. This will be a feature of the starting geometry.

Creation of the starting solid geometry model, therefore, begins with a cylinder that extends from the centerline plane to the edge of the weir. This body is sectioned along its axis by a plane perpendicular to the axis of the trough, and divided by a plane perpendicular to its axis at the inner side of the trough wall. One side of this section will represent the inlet flow region and the other will be the no-slip upper surface of the trough wall.

This cylinder is going to be meshed by sweeping along its axis. As a result, the 90° turn over the weir cannot simply be around a point, but must occur over a small radius of curvature. The flat face of the

half-cylinder geometry is rotated about an axis parallel to the trough axis but shifted by a small amount downwards from the edge the half-cylinder. The result is a 90° elbow with a very small, but finite, inner radius. After this the rotated face of the cylinder is swept downwards the appropriate distance to the exit plane to create an “L” shaped half-cylinder.

Actually, the situation is a bit more complicated than the previous description suggests because prior to sweeping the starting half cylinder, it was cut two more times by planes pass through is axis. These planes sectioned the half-cylindrical shape like a pie with two 45° pieces on opposite sides of a 90° piece. The reason for this step is to provide surfaces that will ultimate become the edge guide features. While the preceding implied that a single face of the half-cylinder was swept rotated and swept again, in reality it was these three pie-shaped face which were manipulated in the same fashion.

A further complication is allowing for mesh fences. Mesh fences will be discussed later on as a means of constraining the mesh motion. At this point, we note that the “L” half-cylinder is further divided by the following planar surfaces: 1) a plane parallel to the vertical trough wall and passing through the turning axis of the 90° elbow, 2) a plane parallel to the top of the trough wall also passing through this axis, 3) a plane which extends from the bottom of the trough, that is, it passes through the eventual detachment plane of the free surface.

The remainder of the geometry falls out naturally. The surfaces that compose the flat faces of the “L” that were at right angles to the original sweeping direction are themselves swept parallel to their normals the appropriate distance. This distance should place the end planes of the new volumes at the centerline of the trough.

The meshing decomposition represents the major creative effort in developing the mesh. Actually creating the mesh is not difficult, but is somewhat tedious. The essential idea is to pave mesh the upper flat faces of the half-cylinder edge feature and sweep these paved meshes through the “L” volume. Care of course is exercised to ensure refinement near important geometry features such as the 90° bend over the weir and the point at which the free-falling film detaches from the trough walls. Once the “L” is meshed, the rest of the domain is meshed by sweeping the z faces of the “L” volume in the negative z direction. Again care is taken to make sure that the bias in the edges of these volumes is such that there is a smooth transition in mesh size from the “L” elements to the rest of the domain.

For future reference the associations of some of the sidesets are given below:

Sideset 35	inlet
Sideset 40	constant z symmetry boundary
Sideset 60	Free Surface
Sideset 20	Planar Edge guide
Sideset 80	Exit Plane
Sideset 10	constant x symmetry boundary
Sideset 30	solid no slip boundaries

and see Figure 2 for more information

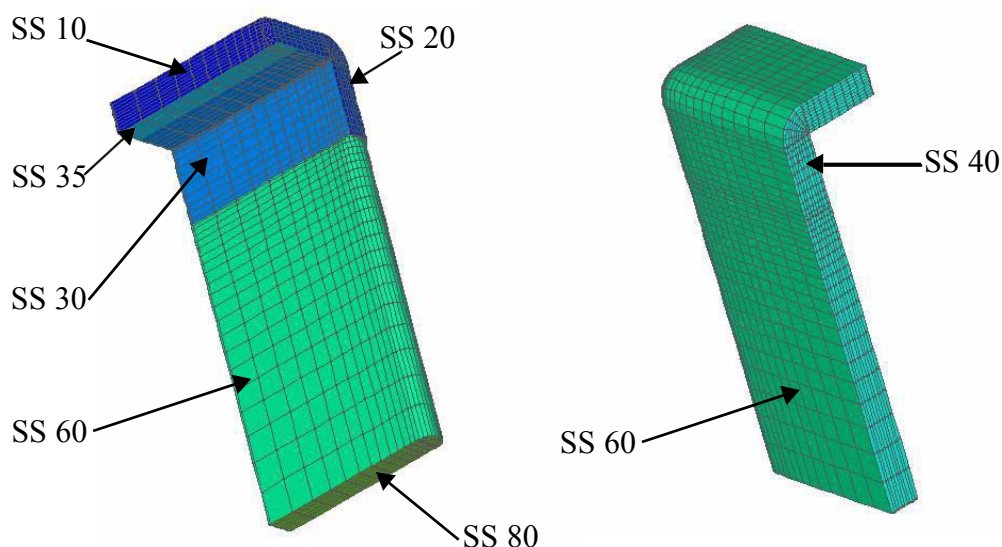


Figure 2. Geometry and sideset location for the weir flow problem

Step 2: Forming the edge guides

The meshed geometry created in the previous step has a smooth starting point associated with free capillary surface. What is missing, however, is the planar edge guides needed to guide the flowing fluid to the detachment point at the bottom of the trough. These are assumed to be vertical, solid, no-slip boundaries that are present at either end of the trough. Their role is to support the extreme ends of the film being formed up until the point where the fluid detaches from the trough and falls freely. We can use *Goma*'s mesh-deforming capability to form these edge guide surfaces in the mesh geometry.

The essential idea is that we set up a problem involving only the three mesh displacement fields and then simply impose the appropriate geometric boundary conditions on the psuedo-solid mesh to deform it into the shape desired. In this case, a PLANE condition is imposed on sideset 20 in order to deform it into the correct planar shape so that it can subsequently serve as the edge guide feature. Other conditions are imposed elsewhere, e.g. PLANE conditions on sideset 40, 80, 35 and 10, that constrain the motion of the mesh as the sideset 20 is being deformed. We also have to include as a user-defined subroutine, a SPLINE condition constraint that is applied to a narrow surface region at the very lip of the weir. This SPLINE constraint pulls the nodes on this surface onto a cylinder with axis parallel to the weir lip and radius equal to the weir lip radius.

Once a convergent result is obtained, the annealed file **anneal.exoII** is moved to the appropriate file name for continuation. In this way, the mesh deformations are locked in and it is just as if the mesh came with edge guides already installed.

The starting geometry, with constructed edge guides, is shown in Figure 2 in a pair of view. An attempt has been made to identify the most prominent sidesets.

Step 3: Computing the fixed mesh velocity solution

As for two-dimensional free surface problems, the next step is to find a velocity/pressure solution for the geometry with the moving free surface boundaries held fixed. On these boundaries a VELO_NORMAL condition is applied to prevent penetration of fluid through them but allow perfect slipping tangent to them. This simulates to a certain extent the boundary condition of an actual free surface. Because the VELO_NORMAL condition is a rotated boundary condition it is necessary to include ROT cards in the input deck. Fortunately, most boundary conditions on the momentum equation are either Dirichlet conditions on the velocity or natural boundary conditions so that the number and complexity of the rotation conditions specified is relatively small. Often it is sometimes only necessary to specify a single ROT card for each of the VELO_NORMAL surfaces.

It is advisable to not include gravitational forces in this stage of the calculation. Since the geometry is rigid, the presence of gravity will produce a negative pressure, “siphoning” pressure inside of the geometry in order to counter-balance the gravitational forces, since the fluid is not permitted to accelerate because of mass balance considerations. When the free surfaces are permitted to move in the next step, the presence of these negative pressures will have a poor effect on the initial iterations. Thus, it is best to avoid them by neglecting gravity.

Usually, it is also best to allow a natural boundary condition at the exit of the domain. At this point the transverse velocities may be fixed at zero, but the axial velocity component should not be set. Again since the geometry is rigid, to do otherwise requires that the exact ratio between inlet and outlet area be determined in order that the outflow velocity is globally consistent with the inlet speed. If this is not the case, no steady solution exists. To avoid, this using a natural boundary condition is recommended.

Step 4: Release the free surface

As for two-dimensional free surface problems, the free surface is released by changing the VELO_NORMAL boundary conditions imposed in the fixed mesh problem to KINEMATIC boundary conditions. The condition is the same, but the latter is imposed on the mesh equations instead of the momentum equations. Consequently, forces will be imposed on the mesh which will distort it to satisfy the kinematic condition instead of pressures on the fluid. One consequence of this is that we have to specify geometric constraints on the other boundaries of the problem, such as a PLANE conditions or Dirichlet conditions on displacements so that the forces being imposed on the free surface are balanced by other forces on the mesh.

Rotation condition specification is more involved for the moving mesh problem because of the presence of numerous strong conditions on the mesh equations which need to be applied correctly in order that each node in the mesh is constrained in the proper way. This document, however, is not a tutorial on application of ROT cards (see Baer (2001)).

The solution from the fixed mesh solution is usually used as the starting guess for the moving mesh problem. There is a difficulty, however, with using this solution and that is the residual pressure field from the fixed mesh solution. As noted above, the fixed boundaries of the domain in this solution result in substantial gradients in the pressure field in the flowwise direction. The boundary conditions

imposed on the free surface for the moving mesh solution do not impose any pressure field on the fluid. This inconsistency can lead to computational difficulties from the beginning unless it is mitigated. A simple solution is to ignore the pressure field resulting from the fixed mesh solution step, and this can be done simply by initializing the pressure field to zero in the input deck of the moving mesh problem. The following card will impose this:

```
Initialize          =  PRESSURE 0 0.0
```

The role of damping should not be minimized with respect to solving three-dimensional free surface problems. By its very nature, it is difficult to creating a starting mesh that is very near to the final result. The best that can be done is creation of a geometry that is topologically similar to the final shape of the free surface. The effect of this is that the starting geometry is not a very good starting guess for the free surface shape on which to apply a Newton-Raphson iterative method. Should one attempt to solve the problem without employing a damping factor, the result would be divergence of the numerical method and the inevitable mesh folding error trap. To prevent this we use a damping factor. Indeed, during the initial iterations, the amount damping is quite restrictive; oftentimes, damping factors on the order of 5% are needed to prevent mesh folding. As the number of iterations progresses and the free surface approaches its final shape, the damping factor can be increased in a likewise manner. Ultimately, it should be possible to set the damping factor to unity and achieve a full Newton-Raphson convergence rate.

Convergence of Iterative Solvers

It goes almost without saying that for all but the smallest of three-dimensional problems, iterative solution of the linear algebra problems is the only viable possibility. The much larger bandwidth simply overwhelms most direct solvers. It is also true that iterative solution methods generally require fewer operations and are faster and thus preferable when dealing with large numbers of unknowns. The primary drawback to using iterative solution methods is lack of robustness. Convergence rates are highly dependent on matrix condition number, and consequently, choice of a good preconditioning method is essential. It is important to recognize that there is no magical iterative solution method or preconditioner. Three dimensional free surface problems, especially, seem to generate problems for all the iterative solvers available to *Goma*. In the next few paragraphs I will describe some of the tricks and techniques I have used on this particular problem to improve the convergence rate of the iterative solvers.

Pressure Stabilization

Pressure stabilization is a technique developed by Hughes, *et.al.* (1986) to remedy somewhat some of the problems with finite element fluid simulation. First, pressure stabilization permits use of equal order pressure and velocity interpolation in apparent violation of the LBB stability criterion. This is critically important three dimensional problems since it permits Q1/Q1 interpolation on eight-node hex elements. Instead of Q2/Q1, which is horrendously expensive, or Q1/P0 which requires a special nine-noded element. The other benefit of pressure stabilization is that by adding the residual of the momentum equation to the continuity equation (weighted by a small, adjustable parameter, of course) a diagonal element appears in the matrix equations for these equations. This improves tremendously

the matrix condition number and actually allows for the use of iterative solvers, which otherwise would be unable to solve the matrix.

The problem that arises with pressure stabilization is that we do not add the complete residual of the momentum equation. Instead, because of shape function continuity issues, it is necessary to exclude the quadratic viscous term in the momentum residual added to the continuity equation. This means that even in the converged limit a mass error will be present in the continuity equation. The size of this error is controlled by the size of the pressure stabilization weighting parameter.

The latter problem, however, is most prominent in interior flows with strong gradients in pressure. In our particular case, we are solving a problem in which the viscous forces balance the gravitational forces primarily and there is very little gradients in pressure. Moreover, the viscous forces tend to dominate the gravitational forces resulting in a small Laplacian viscous term, which is the term excluded from the pressure stabilization terms. The result is that the size of the pressure stabilization parameter introduces a relatively small global mass balance error, but at the same time providing conditioning to the matrix problem and improved iterative solver performance. We found that pressure stabilization parameters on the order of 1.0 had little effect on the global mass conservation, but considerable effect on the solvers.

Mesh Scaling

Another idea employed to improve convergence rates for the iterative solver was mesh scaling. This is a very simple technique in which all three linear dimensions of the mesh are scaled down by a common factor; we used 10.0, for example. All other parameters retain their same scaling; it is only distances that are changed. One can think of this as measuring distances in meters instead of centimeters, for example. It is also important in the formulation that each term in the equation be multiplied by the appropriate power of this scaling factor. For example, the advective terms must be multiplied by the factor itself, while diffusive terms should be multiplied by the square of the scaling factor. It is probably easiest to do this when the equations are cast into dimensionless terms. That way all that needs to be done is to multiply the dimensional distance scale by the scaling factor to accomplish the proper weight of terms.

The reason why this type of scaling tends to improve the behavior of the iterative solvers has to do with the KINEMATIC boundary condition. This boundary condition is the bad actor because it replaces a component of the mesh equations but is only weakly dependent on the mesh displacement. This creates another set of equations (in addition to the continuity equation) which is not diagonally dominant and degrades the matrix condition number. If we look at the sensitivity of the KINEMATIC constraint with respect to mesh displacement, we find it depends on a term of the form,

$$\frac{\partial n_{\alpha}}{\partial d_{\beta}} \propto \frac{1}{A_{surf}^2} \quad (1)$$

where A_{surf} is the local element surface area. As can be seen, if this area is decrease by scaling, the effect on this critical sensitivity will be enhance disproportionately. This results in a better conditioned matrix.

Rotation conditions

A key element in generating a well-conditioned matrix is a complete and consistent set of rotation conditions. It is not the intent of this document to present a tutorial of the proper application of rotation conditions; such a tutorial already exists (see Baer (2001)). We will, however, describe how through clever use of rotation conditions the condition of the matrix can be improved. The problem centers around the conformation of the free surface sideset, which in our mesh we have designated 60. A KINEMATIC boundary condition is applied to this surface, which, as noted above, has poor effects on the matrix condition number. An additional problem is that on the upper surface nearest to the inlet of the problem this constraint is primarily sensitive to displacements in the y direction. Therefore, according to our rules for assigning rotation conditions, the proper rotation condition would be:

```
ROT = MESH SURFACE 60 T1 0 KINEMATIC 60 T2 0 SEED 1 -1 0
```

However, over much of the rest of the free surface, this boundary condition is sensitive primarily to displacements in the x direction, so that in these regions we would need this rotation condition:

```
ROT = MESH SURFACE 60 KINEMATIC 60 T1 0 T2 0 SEED 1 -1 0
```

We can only have one rotation condition per surface sideset but by ignoring one of these conditions we automatically degrade our matrix condition. What to do ?

The answer is, of course, simple: define a new sideset. However, implementing this idea is a little subtle. Figure 3 shows the location of sideset 64, the new sideset set we have defined. It encompasses the upper surface of the weir's free surface, but does not extend past the weir lip. Now if we use two rotation conditions in the following order:

```
ROT = MESH SURFACE 64 T1 0 KINEMATIC 64 T2 0 SEED 1 -1 0
```

```
ROT = MESH SURFACE 60 KINEMATIC 60 T1 0 T2 0 SEED 1 -1 0
```

then the KINEMATIC boundary condition on the upper free surface will be associated with the correct degree of freedom. Order is important here because for a node to which more than one surface ROT card could apply, the choice always goes to the ROT card that appears first in the input deck. In this case, nodes that belong to sideset 64 also belong to sideset 60, but will have the first ROT card applied to them.

The subtle part appears when considering the boundary between sidesets 64 and 60. Left alone nodes on this boundary would have two KINEMATIC boundary conditions applied to them, one from sideset 60 on the x equation on one from sideset 64 on the y equation. This is not correct. To fix this, we apply the following EDGE card:

```
ROT = MESH EDGE 70 60 PLANEX 70 KINEMATIC 60 T 0 NONE
```

Because sideset 60 is present on both sides of this particular EDGE, the nodes on this edge will receive contributions from the elements both upstream and downstream of the edge as desired. Note

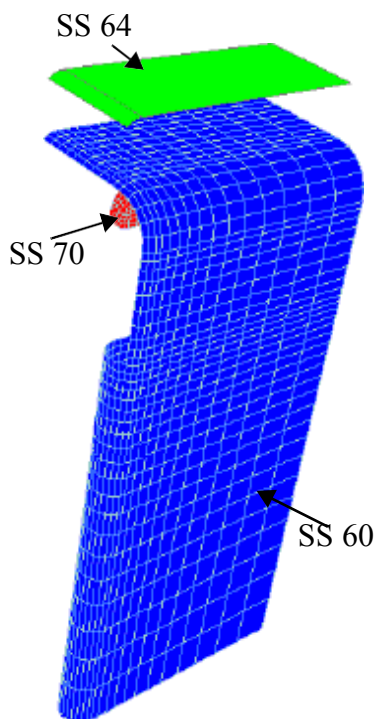


Figure 3. Location of new sideset 64 in weir flow problem.

the “mesh fence” sideset 70 which actually is an interior sideset and is conveniently located for defining this edge.

Deformed geometry remeshing with *Cubit*

A newly implemented capability in the *Cubit* meshing tool is the ability to construct a mesh on a deformed geometry. *Cubit* constructs what is referred to as a “faceted” representation of the geometry based upon the surface finite elements in the deformed geometry. This “faceted” solid model functions in the same way as the original ACIS solid model constructed prior to creation of the starting finite element mesh. It defines the shape of the geometry and the decomposition used to create a mesh. In practice, one uses the “faceted” solid model in exactly the same way as the original ACIS solid model, that is, there are volumes, surfaces, curves and vertices and one would mesh the deformed representation just as if you were meshing the original ACIS representation.

There are, however, a couple of important points. First, the current implementation of the faceted solid model does not permit the so-called “Boolean” operations. Things like webcutting, subtracting one volume from another or uniting one volume with another are proscribed¹ for the deformed geometry representation. This means that you cannot decompose the deformed representation further to make it easier to mesh. If you imagine for the moment attempting to create a mesh on the deformed geometry of the weir flow present previously, you recognize that to be unable to decompose it effectively prevents you from doing so. This leads to the second point. To preserve the original meshing decomposition to remesh the deformed geometry, it is necessary to define initially, before any *Goma* computations are performed, a separate element block for each of the volumes in the original solid geometry meshing decomposition. This is in contrast to usual situation in which once

1. proscribe means you can't do it.

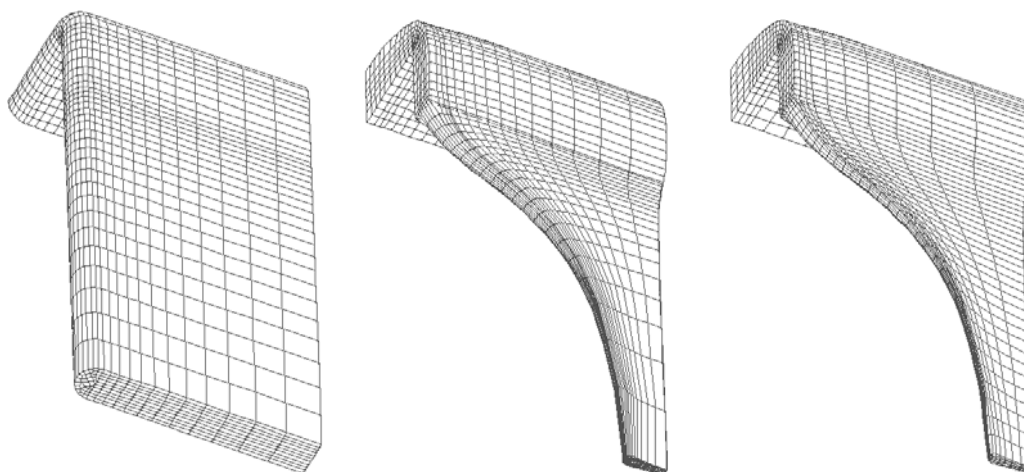


Figure 4. Undeformed weir flow geometry (left), Deformed solution prior to remeshing (center), Deformed solution after remeshing (right)

the original geometry is meshed, the following *Cubit* command is issued to lump all the meshing volumes into a single element block:

```
CUBIT> vol all element block 1
```

In order to permit deformed geometry remeshing later on, it is necessary to replace this command with the following set of commands:

```
CUBIT> vol 1 element block 1
CUBIT> vol 2 element block 2
CUBIT> vol 3 element block 3
CUBIT> vol 4 element block 4
.
.
.
```

(one is of course free to choose the element block ids as one sees fit. An ascending list of ordered integers is not required.)

With this done, *Cubit* is able to retain the meshing decomposition when it constructs the facted geometry model from the deformed geometry information. The only change this necessitates in the *Goma* input file is that a list of these element blocks must be present after the material file designation, for example:

```
MAT = newt 1 2 3 4 5 6
```

The bad news about this is that existing deformed geometry files which did not retain the meshing decomposition via this procedure *cannot* be remeshed by *Cubit* at this time.

Having created a multi-block mesh for later deform remeshing, *Goma* is applied to it to solve the flow problem and deform the geometry appropriately. At some point during this process, you will want to remesh the geometry. Assume the exodus file with the deformed solution is saved to "deformed.exoII." *Cubit* is launched and the following two commands are executed:

```
import mesh geometry "deformed.exoII" feature_angle 135 deformed last  
delete mesh volume all propagate
```

This imports the geometry from the exodus file and deletes the preexisting mesh. What remains is the original set of element blocks, now deformed, as meshable volumes. One now proceeds as you would when meshing any other solid body geometry model. The only caveat is that it is possible that if oscillations are present in the free surface, it may not be possible for *Cubit* to mesh the deformed element block. Its best to remesh before that occurs.

Once a mesh is created on the deformed geometry, the solution fields from the original file must be mapped onto the new geometry. As with two-dimensional remeshing, this is done with the SEAMS tool MAPVAR. This document is not intended as a tutorial on operation of MAPVAR. For those users familiar with its use from two-dimensional remeshing, it functions in precisely the same manner in this case.

With the old solution now mapped onto the new mesh, it is possible to resume the computation. Normally it is best to resolve the case just remeshed at the same parameter values. MAPVAR afterall interpolates field variables at nodes on the new mesh. This will serve to push the solution slightly away from the original converged point in the degree of freedom N space. Solving again at the same parameters moves the solution back to this point.

Figure 4 shows the sequence. The leftmost figure is the original undeformed mesh fresh from CUBIT. The central figure is the converged solution on the old mesh. The rightmost figure shows the converged solution at the same parameters on the new mesh. Although the last two figures, look suspiciously like the same mesh, there are key differences. Notice, for example, that at the exit plane there are now more elements in the streamwise direction than in the original mesh.

References

1. Hughes, T. R. J., L. P. Franca and M. Balestra, "A New Finite Element Formulation for Computational Fluid Dynamics: V. Circumventing the Babuska-Brezzi Condition: A Stable Petrov-Galerkin Formulation of the Stokes Problem Accomodating Equal-Order Interpolations," Comput. Methods Appl. Mech. Engrg., 59 (1986) 85-99.
2. Baer, T.A., "ROT card tutorial (GT-018.1)," Internal Sandia memo, January 22, 2001.

Appendix A: Files and usage for weir problem

Cubit and Geometry files:

big_weir_acis.jou - *Cubit* journal file which creates the solid body geometry of the starting weir flow geometry. This is written to the file **big_weir.sat**.

big_weir.sat - This is the ACIS geometry file created by *Cubit*. It has been included for convenience.

big_weir.jou - *Cubit* journal file which imports the ACIS geometry file created by **big_weir_acis.jou** and constructs a mesh. The mesh is created in a file named **big_weir.exoII**

Mesh-only files

big_weir_mesh.inp - *Goma* input file that solves for mesh motion only. This is used prior to any other computations to create the flat edge guide geometry that could not be created by *Cubit*.

Fixed mesh files:

big_weir_fixed.inp - *Goma* input file for solving the velocity/pressure flow problem only, i.e. no mesh motion

big_weir.defs - file containing flow and material parameters as appropriate definitions.

newt.mat - material file defining the material properties of the fluid.

Moving mesh files:

big_weir.inp - *Goma* input deck for solving the full-up, moving mesh calculation.

Parallel computing:

brk_udp.in - input file for the domain decomposition *brk* specialized for the moving mesh problem.

Source

user_bc.c - This user-subroutine contains specific geometry functions that are needed when solving the mesh-only problem (see below).

Obtaining a solution (serial)

The first step is creating the mesh in *Cubit*. This is done by running *Cubit* with **big_weir.jou** journal file as input. The resulting mesh will be written to **big_weir.exoII**.

The mesh just created, however, lacks the proper shape for the edge guide geometry. To remedy this, *Goma* is used to deform the mesh appropriately and make the proper geometry. This step requires that the specific geometry functions present in **user_bc.c** be compiled and linked to the *Goma*

executable. These geometry functions define the narrow cylindrical surface at the lip of the weir where the fluid makes a right angle turn and starts to descend under gravity.

One should copy the exodus file `big_weir.exoII` into `contin.exoII` prior to executing *Goma* with the following command

```
% goma -i big_weir_mesh.inp -a
```

After several iterations a convergent answer will be obtained. Copy the `anneal.exoII` file into `contin.exoII`. It is also probably wise to copy into a different filename in case a restart from this point is needed.

The next step is to solve the fixed mesh problem to obtain a starting velocity/pressure field. This is done by simply invoking *Goma* with the command:

```
goma -i big_weir_fixed.inp -a
```

In this case, the fluid is relatively viscous so that this is almost a linear problem and convergence should be achieved in only a few iterations. At the conclusion of this computation copy the output exodus file, `out.exoII`, to `contin.exoII` in preparation for the next stage in the computation.

The next step is allowing the mesh to move. With the fixed mesh solution copied into `contin.exoII`, this is done with the following command:

```
goma -i big_weir.inp -a -r 0.05 -n 25
```

As usual we are using quite a bit of damping. Note also in the file `big_weir.inp`, the `PRESSURE` field is initialized to a uniform zero value. The reason is to eliminate the “siphoning” pressures present as a result of solving the problem on a fixed mesh. This initialization is appropriate only when starting from the fixed mesh solution. If restart with a different `contin.exoII`, be sure to comment out this card.

You might have to run *Goma* in this way for several of these damped iteration cycles, each with larger and larger damping coefficients, before it will be possible to turn off damping altogether and proceed with full Newton-Raphson iteration to the final result.

Obtaining a solution (parallel)

Solving the moving mesh problem using more than one processor follows the preceding with a couple of pre and post steps thrown in.

To begin with you must rebuild your *Goma* executable with the `PARALLEL` compiler flag set in the makefile. You must also link to MPI by making sure the `MPI_TOP`, `MPI_INC`, and `MPI_LIB` makefile variables are uncommented and point to the right locations in your file system. Also there is a special AZTEC library for parallel operations, so make sure that the `AZTEC_LIB` variable includes `-laztec-mpi` in place of `-laztec`.

Next you have to construct a machinelist file. This is nothing more than a text file with the hostnames of the processors on which you want your job partition. For multi-processor hosts you repeat the hostname for as many processors you want committed. For example, if you have two single processor boxes with hostnames, tom and dick, and another four processor box with hostname, harry and you want to use all six processors, your machinelist file would look like:

```
tom
dick
harry
harry
harry
harry
```

The next step is to decompose the domain into pieces. This is accomplished using a utility called brk, which should be included with your distribution (you will have to make the executable if you haven't already). For example, to decompose the database file contin.exoII into 6 pieces for six processors invoke the following:

```
brk -n 6 brk_udp.in contin.exoII
```

when this finishes you will see size new exodus files with names like contin_1of6.exoII, contin_2of6.exoII now present in your directory. You can blot these to view how the decomposition was done, which is interesting.

Next you run *Goma* inside a the MPI parallel shell called mpirun:

```
mpirun -machinefile machinelist -np 6 ./goma -i weir.inp -a -r 0.1 -n 25
```

The file machinelist is the machinelist file you just created. In this case, you should also copy the *Goma* executable into your run directory or create a link to the executable in your directory.

When this finishes executing you will note that there are now a bunch of files with names like out_1of6.exoII, out_2of6.exoII, etc., which are, as you might imagine are associated with the original contin.exoII pieces and are the solution database on each of the decomposed pieces. You need to reconstitute the “monolithic” exodus database through use of another utility called fix:

```
fix -n 6 out
```

This utility produces the single file out.exoII which you can copy to contin.exoII to restart the problem (after breaking contin.exoII up once again using brk).